

Finanças — Teoria Completa com Python

Capítulo 1 — Introdução às Finanças

1.1. Definição e Escopo

Finanças é a arte e a ciência de administrar o dinheiro. Mais precisamente, é o campo do conhecimento que estuda como os agentes econômicos (indivíduos, empresas, governos) alocam recursos escassos ao longo do tempo, considerando riscos e incertezas.

A área de finanças se divide em quatro grandes ramos:

Ramo	Objeto de Estudo
Finanças Corporativas	Decisões financeiras dentro das empresas (investir, financiar, distribuir)
Investimentos	Alocação de recursos em ativos financeiros (ações, títulos, derivativos)
Instituições Financeiras	Bancos, corretoras, seguradoras, fundos — seu papel e regulação
Finanças Internacionais	Fluxos financeiros entre países, câmbio, risco-país

1.2. As Três Decisões Fundamentais

O gestor financeiro de uma empresa enfrenta três grandes decisões, que formam o tripé das finanças corporativas:

Decisão de Investimento (Capital Budgeting)

Onde aplicar os recursos? Quais projetos, máquinas, aquisições ou ativos gerarão o maior retorno ajustado ao risco.

Envolve: - Estimativa de fluxos de caixa futuros - Análise de risco do projeto - Métricas de avaliação: VPL, TIR, Payback, IL - Priorização entre projetos concorrentes

Decisão de Financiamento (Estrutura de Capital)

Como captar os recursos? Qual a combinação ideal entre capital próprio (ações, lucros retidos) e capital de terceiros (dívidas, debêntures, empréstimos).

Envolve: - Custo de cada fonte de capital - WACC (Custo Médio Ponderado de Capital) - Teoria de Modigliani-Miller - Trade-off entre risco e benefício fiscal

Decisão de Dividendos

Quanto distribuir aos acionistas vs reinvestir? Qual a política ótima de distribuição de resultados.

Envolve: - Pay-out ratio (percentual do lucro distribuído) - Dividend Yield - Recompra de ações - Teoria da irrelevância dos dividendos

1.3. Objetivo da Empresa

O objetivo consensual da teoria financeira moderna é **maximizar a riqueza dos acionistas** (shareholder wealth maximization). Isso se traduz em maximizar o preço das ações no longo prazo.

Por que não maximizar o lucro? - O lucro contábil é uma medida de curto prazo - O lucro não considera o risco - O lucro não considera o valor do dinheiro no tempo - O lucro pode ser manipulado por práticas contábeis

Valor da empresa:

$$Valor = \sum_{t=1}^{\infty} \frac{FCL_t}{(1 + WACC)^t}$$

Onde FCL_t são os fluxos de caixa livres gerados em cada período.

```
def valor_descontado(fluxos, wacc):  
    """Calcula o valor presente de uma série de fluxos de caixa."""  
    return sum(f / (1 + wacc) ** (t + 1) for t, f in enumerate(fluxos))  
  
fluxos = [100, 115, 132, 152, 175]  
wacc = 0.12  
v = valor_descontado(fluxos, wacc)  
print(f"Valor presente dos fluxos (5 anos): R${v:.2f} milhões")
```

1.4. Gestão Baseada em Valor (VBM)

Value-Based Management é uma filosofia de gestão que alinha todas as decisões — estratégicas, operacionais e financeiras — à criação de valor para o acionista.

Métricas-chave do VBM:

Métrica	Fórmula	Interpretação
EVA (Economic Value Added)	NOPAT - (Capital × WACC)	Lucro econômico real
MVA (Market Value Added)	Valor de Mercado - Capital Investido	Riqueza criada para o acionista
ROIC	NOPAT / Capital Investido	Retorno sobre o capital
ROE	Lucro Líquido / Patrimônio Líquido	Retorno sobre capital próprio

Regra fundamental: Se $ROIC > WACC$, a empresa está **criando valor**. Se $ROIC < WACC$, está **destruindo valor**.

```
def metricas_vbm(nopat, capital_investido, wacc, valor_mercado, ll, pl):  
    eva_ = nopat - (capital_investido * wacc)  
    mva_ = valor_mercado - capital_investido  
    roic = nopat / capital_investido  
    roe = ll / pl  
    return {'EVA': eva_, 'MVA': mva_, 'ROIC': roic, 'ROE': roe}  
  
exemplo = metricas_vbm(500, 4000, 0.12, 6500, 350, 3000)  
for k, v in exemplo.items():  
    if k in ('ROIC', 'ROE'):  
        print(f"{k}: {v*100:.2f}%")  
    else:  
        print(f"{k}: R${v:.2f}")
```

Capítulo 2 — Risco e Retorno

2.1. Conceitos Fundamentais

Retorno é o ganho ou perda de um investimento em determinado período. Pode vir de duas fontes:

- **Ganho de capital:** variação no preço do ativo ($P_t - P_{t-1}$)
- **Renda periódica:** dividendos, juros, aluguéis (D_t)

$$R_t = \frac{P_t - P_{t-1} + D_t}{P_{t-1}}$$

Risco é a probabilidade de o retorno real diferir do retorno esperado. Quanto maior a dispersão dos possíveis retornos, maior o risco.

Métricas de risco: - **Variância:** $\sigma^2 = E[(R - E(R))^2]$ - **Desvio padrão:** $\sigma = \sqrt{\sigma^2}$ (mede o risco total) - **Semidesvio:** considera apenas desvios negativos (downside risk) - **Value at Risk (VaR):** perda máxima esperada em dado nível de confiança

```
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from scipy import stats

# Baixar dados
ticker = "PETR4.SA"
dados = yf.download(ticker, start="2020-01-01", end="2024-01-01")
precos = dados['Adj Close']
retornos = precos.pct_change().dropna()

# Estatísticas
print(f"{'Métrica':<25} {'Valor':<15}")
print("-" * 40)
print(f"{'Retorno médio diário':<25} {retornos.mean()*100:.4f}%")
print(f"{'Volatilidade diária':<25} {retornos.std()*100:.4f}%")
print(f"{'Volatilidade anualizada':<25} {retornos.std() * np.sqrt(252) * 100:.2f}%")
print(f"{'Máximo retorno diário':<25} {retornos.max()*100:.2f}%")
print(f"{'Mínimo retorno diário':<25} {retornos.min()*100:.2f}%")
print(f"{'Assimetria (skewness)':<25} {retornos.skew():.4f}")
print(f"{'Curtose (kurtosis)':<25} {retornos.kurtosis():.4f}")
print(f"{'VaR 95% (histórico)':<25} {retornos.quantile(0.05)*100:.2f}%")
print(f"{'VaR 99% (histórico)':<25} {retornos.quantile(0.01)*100:.2f}%")
```

2.2. Retorno Esperado

Quando não temos dados históricos, usamos cenários probabilísticos:

$$E(R) = \sum_{i=1}^n p_i \times R_i$$

Onde p_i é a probabilidade do cenário i ocorrer e R_i é o retorno nesse cenário.

```
def retorno_esperado(cenarios):
    """
    Calcula retorno esperado a partir de cenários.
    cenarios: lista de (probabilidade, retorno)
    """
    return sum(p * r for p, r in cenarios)

def risco_total(cenarios):
    """Calcula desvio padrão a partir de cenários."""
    er = retorno_esperado(cenarios)
    variancia = sum(p * (r - er) ** 2 for p, r in cenarios)
```

```

    return np.sqrt(variancia)

# Cenários econômicos para uma ação
cenarios = [
    (0.15, 0.35), # boom: 15% chance, 35% retorno
    (0.30, 0.18), # expansão: 30% chance, 18% retorno
    (0.30, 0.08), # normal: 30% chance, 8% retorno
    (0.15, -0.05), # recessão: 15% chance, -5% retorno
    (0.10, -0.20) # crise: 10% chance, -20% retorno
]

er = retorno_esperado(cenarios)
sigma = risco_total(cenarios)
print(f"Retorno esperado: {er*100:.2f}%")
print(f"Risco (desv. pad.): {sigma*100:.2f}%")
print(f"Relação retorno/risco: {er/sigma:.2f}")

```

2.3. Risco de uma Carteira (Portfólio)

O risco de uma carteira **não** é a média ponderada dos riscos individuais, pois a **correlação** entre os ativos reduz (ou aumenta) o risco total.

Retorno da carteira:

$$E(R_p) = \sum_{i=1}^n w_i \times E(R_i)$$

Variância da carteira (2 ativos):

$$\sigma_p^2 = w_1^2 \sigma_1^2 + w_2^2 \sigma_2^2 + 2w_1 w_2 \sigma_1 \sigma_2 \rho_{12}$$

Variância da carteira (n ativos):

$$\sigma_p^2 = \sum_{i=1}^n \sum_{j=1}^n w_i w_j \sigma_{ij}$$

Onde $\sigma_{ij} = \rho_{ij} \times \sigma_i \times \sigma_j$ é a covariância.

```

def risco_carteira(pesos, cov_matrix):
    """
    Calcula o risco (desvio padrão) de uma carteira.
    pesos: array de pesos dos ativos
    cov_matrix: matriz de covariância
    """
    return np.sqrt(pesos @ cov_matrix @ pesos)

def simulacao_diversificacao():
    """
    Demonstra o efeito da diversificação com N ativos.
    """
    np.random.seed(42)
    risco_comum = 0.40
    correlacao = 0.15 # correlação média entre ativos

    for n_ativos in [1, 2, 5, 10, 20, 50, 100]:
        if n_ativos == 1:
            risco_total = risco_comum
        else:

```

```

risco_nao_sistematico = risco_comum / np.sqrt(n_ativos)
risco_sistematico = risco_comum * np.sqrt(correlacao)
risco_total = np.sqrt(risco_sistematico**2 + risco_nao_sistematico**2)
print(f"N = {n_ativos:3d} ativos -> Risco da carteira:
{risco_total*100:.2f}%")

```

```
simulacao_diversificacao()
```

```

N = 1 ativos -> Risco da carteira: 40.00%
N = 2 ativos -> Risco da carteira: 28.54%
N = 5 ativos -> Risco da carteira: 20.19%
N = 10 ativos -> Risco da carteira: 17.06%
N = 20 ativos -> Risco da carteira: 15.82%
N = 50 ativos -> Risco da carteira: 15.25%
N = 100 ativos -> Risco da carteira: 15.11%

```

Conclusão fundamental: A diversificação reduz o risco, mas há um limite mínimo — o **risco sistemático** (de mercado) — que não pode ser eliminado por mais que se diversifique.

Risco Total = Risco Sistemático (mercado) + Risco Não-Sistemático (empresa)

2.4. CAPM — Capital Asset Pricing Model

Desenvolvido por Sharpe (1964), Lintner (1965) e Mossin (1966), o CAPM é o modelo mais influente para precificação de ativos financeiros.

Premissas do CAPM: 1. Investidores são racionais e avessos ao risco 2. Mercados são perfeitos (sem custos de transação, sem impostos) 3. Todos os investidores têm o mesmo horizonte de investimento 4. Todos têm as mesmas expectativas sobre retornos e riscos 5. Existe um ativo livre de risco (R_f) 6. Todos podem tomar emprestado à taxa livre de risco

Equação do CAPM:

$$E(R_i) = R_f + \beta_i \times [E(R_m) - R_f]$$

Onde: - R_f = taxa livre de risco (Selic, Treasury bond) - $E(R_m)$ = retorno esperado da carteira de mercado - $[E(R_m) - R_f]$ = prêmio de risco do mercado - β_i = sensibilidade do ativo i aos movimentos do mercado

```

def capm(rf, beta, premio_mercado):
    return rf + beta * premio_mercado

```

```

rf = 0.105      # 10.5% (Selic real)
premio = 0.055 # 5.5% prêmio histórico do Ibovespa

```

```

print(f"Selic (Rf): {rf*100:.1f}%")
print(f"Prêmio de risco: {premio*100:.1f}%\n")
print(f"{'Beta':<8} {'Retorno Esperado':<20} {'Perfil'}")
print("-" * 45)
for beta in [0.0, 0.5, 0.8, 1.0, 1.2, 1.5, 2.0]:
    ret = capm(rf, beta, premio) * 100
    perfil = {0: 'Livre de risco', 0.5: 'Defensivo', 0.8: 'Moderado',
              1.0: 'Mercado', 1.2: 'Agressivo', 1.5: 'Agressivo+', 2.0: 'Alavancado'}
    print(f"{'beta':<8.1f} {'ret':<19.2f}% {'perfil.get(beta, '')}")

```

2.5. Beta (β)

O beta mede o **risco sistemático** de um ativo:

$$\beta_i = \frac{Cov(R_i, R_m)}{Var(R_m)}$$

Interpretação: - $\beta = 1$: o ativo acompanha o mercado - $\beta > 1$: o ativo é mais volátil que o mercado (ações de crescimento, small caps) - $0 < \beta < 1$: o ativo é menos volátil que o mercado (utilidades, defensivas) - $\beta = 0$: o ativo não tem correlação com o mercado (caixa) - $\beta < 0$: o ativo se move na direção oposta ao mercado (ouro, alguns hedge funds)

```
def calcular_beta(dados_acao, dados_mercado):
    """
    Calcula o beta de uma ação contra o mercado.
    """
    r_acao = dados_acao.pct_change().dropna()
    r_mercado = dados_mercado.pct_change().dropna()
    dados = pd.concat([r_acao, r_mercado], axis=1, join='inner').dropna()
    cov = dados.iloc[:, 0].cov(dados.iloc[:, 1])
    var_mercado = dados.iloc[:, 1].var()
    beta = cov / var_mercado

    # Regressão linear para informações adicionais
    from scipy import stats
    slope, intercept, r_value, p_value, std_err = stats.linregress(
        dados.iloc[:, 1], dados.iloc[:, 0]
    )

    return {
        'beta': beta,
        'alpha': intercept,
        'r_quadrado': r_value ** 2,
        'p_valor': p_value
    }

# Exemplo de uso (com dados reais)
# acao = yf.download("PETR4.SA")['Adj Close']
# mercado = yf.download("^BVSP")['Adj Close']
# resultado = calcular_beta(acao, mercado)
# print(f"Beta: {resultado['beta']:.2f}")
# print(f"R²: {resultado['r_quadrado']:.3f}")
```

2.6. Críticas e Limitações do CAPM

1. **Premissas irreais:** mercados não são perfeitamente eficientes
2. **Beta não é estável:** muda ao longo do tempo
3. **Proxy de mercado:** o Ibovespa (ou S&P 500) não é a “carteira de mercado” teórica
4. **Anomalias:** fatores tamanho, valor, momentum explicam retornos melhor que o beta sozinho

Alternativas ao CAPM: - **Modelo de 3 Fatores de Fama-French:** adiciona SMB (small minus big) e HML (high minus low) - **Modelo de 4 Fatores (Carhart):** adiciona momentum - **Modelo de 5 Fatores (Fama-French):** adiciona profitability e investment - **APT (Arbitrage Pricing Theory):** múltiplos fatores macroeconômicos

2.7. Fronteira Eficiente de Markowitz

Harry Markowitz (1952) demonstrou que existe um conjunto de carteiras que oferecem o **máximo retorno para cada nível de risco** — a **fronteira eficiente**.

Problema de otimização: - Dado um conjunto de ativos com retornos esperados, variâncias e covariâncias - Encontrar os pesos w_i que minimizam o risco para cada nível de retorno

```
def fronteira_eficiente(retornos, cov_matrix, n_pontos=100):
    """
    Calcula a fronteira eficiente de Markowitz.
    """
    n = len(retornos)
    resultados = {'risco': [], 'retorno': [], 'sharpe': [], 'pesos': []}

    # Gera carteiras aleatórias
    for _ in range(10000):
        w = np.random.random(n)
        w = w / w.sum()
        ret = w @ retornos
        risco = np.sqrt(w @ cov_matrix @ w)
        sharpe = ret / risco if risco > 0 else 0
        resultados['risco'].append(risco)
        resultados['retorno'].append(ret)
        resultados['sharpe'].append(sharpe)
        resultados['pesos'].append(w)

    return resultados

def carteira_otima_sharpe(resultados):
    """Encontra a carteira com maior Índice de Sharpe."""
    idx = np.argmax(resultados['sharpe'])
    return {
        'retorno': resultados['retorno'][idx],
        'risco': resultados['risco'][idx],
        'sharpe': resultados['sharpe'][idx],
        'pesos': resultados['pesos'][idx]
    }

def carteira_minima_volatilidade(resultados):
    idx = np.argmin(resultados['risco'])
    return {
        'retorno': resultados['retorno'][idx],
        'risco': resultados['risco'][idx],
        'sharpe': resultados['sharpe'][idx],
        'pesos': resultados['pesos'][idx]
    }

# Simulação com 4 ativos
retornos_esperados = np.array([0.12, 0.15, 0.09, 0.14])
cov_matrix = np.array([
    [0.04, 0.012, 0.008, 0.015],
    [0.012, 0.09, 0.01, 0.025],
    [0.008, 0.01, 0.03, 0.009],
    [0.015, 0.025, 0.009, 0.07]
])

resultados = fronteira_eficiente(retornos_esperados, cov_matrix)
max_sharpe = carteira_otima_sharpe(resultados)
min_vol = carteira_minima_volatilidade(resultados)

print("Carteira de Máximo Sharpe:")
```

```

print(f" Retorno: {max_sharpe['retorno']*100:.2f}%")
print(f" Risco: {max_sharpe['risco']*100:.2f}%")
print(f" Sharpe: {max_sharpe['sharpe']:.3f}")

print("\nCarteira de Mínima Volatilidade:")
print(f" Retorno: {min_vol['retorno']*100:.2f}%")
print(f" Risco: {min_vol['risco']*100:.2f}%")
print(f" Sharpe: {min_vol['sharpe']:.3f}")

```

Capítulo 3 – Custo de Capital

3.1. Conceito

O **custo de capital** é a taxa de retorno mínima exigida pelos provedores de capital (acionistas e credores) para investir em uma empresa. É a **TMA** (Taxa Mínima de Atratividade) para novos projetos.

“O custo de capital é a taxa de retorno que uma empresa precisa obter sobre seus investimentos para manter o valor de suas ações inalterado.” — Brigham & Ehrhardt

3.2. Custo de Capital Próprio (Ke)

É a taxa de retorno exigida pelos acionistas. As principais formas de estimá-lo:

1. CAPM:

$$Ke = R_f + \beta \times (R_m - R_f)$$

2. Modelo de Gordon (Dividend Discount Model):

$$Ke = \frac{D_1}{P_0} + g$$

3. Bond Yield + Risk Premium:

$$Ke = Kd + \text{Prêmio}$$

```

def custo_capital_proprio_capm(rf, beta, rm):
    return rf + beta * (rm - rf)

def custo_capital_proprio_gordon(dividendo_proximo, preco_atual, crescimento):
    return dividendo_proximo / preco_atual + crescimento

# CAPM
rf, beta, rm = 0.105, 1.1, 0.165
ke_capm = custo_capital_proprio_capm(rf, beta, rm)

# Gordon
d1, po, g = 2.50, 45.00, 0.04
ke_gordon = custo_capital_proprio_gordon(d1, po, g)

print(f"Ke (CAPM): {ke_capm*100:.2f}%")
print(f"Ke (Gordon): {ke_gordon*100:.2f}%")

```

3.3. Custo de Capital de Terceiros (Kd)

É a taxa efetiva que a empresa paga sobre suas dívidas. Como os juros são dedutíveis do IR, usa-se o **custo líquido**:

$$Kd_{líquido} = Kd_{bruto} \times (1 - IR)$$

Formas de estimar: - Taxa de juros dos empréstimos bancários recentes - Yield to maturity (YTM) das debêntures da empresa - Spread sobre o CDI ou Selic

```
def custo_terceiros(kd_bruto, aliquota_ir):
    return kd_bruto * (1 - aliquota_ir)

# Empresa paga CDI + 2,5% a.a. com CDI a 13,25%
cdi = 0.1325
spread = 0.025
kd_bruto = (1 + cdi) * (1 + spread) - 1

print(f"Custo bruto da dívida: {kd_bruto*100:.2f}% a.a.")
print(f"Alíquota de IR: 34%")
print(f"Custo líquido da dívida: {custo_terceiros(kd_bruto, 0.34)*100:.2f}% a.a.")
```

3.4. WACC — Weighted Average Cost of Capital

O WACC é a média ponderada do custo de cada fonte de capital:

$$WACC = \frac{E}{V} \times Ke + \frac{D}{V} \times Kd \times (1 - IR)$$

Onde: - E = valor de mercado do capital próprio (equity) - D = valor de mercado da dívida (debt) - $V = E + D$ = valor total da empresa

```
def wacc(ke, kd_bruto, ir, peso_pl, peso_divida):
    kd_liquido = kd_bruto * (1 - ir)
    return peso_pl * ke + peso_divida * kd_liquido

def analise_wacc():
    """
    Análise de sensibilidade do WACC a mudanças na estrutura de capital.
    """
    ke = 0.145
    kd_bruto = 0.12
    ir = 0.34

    print(f"{'Dívida (%)':<12} {'PL (%)':<12} {'WACC (%)':<12}")
    print("-" * 36)
    for p_divida in np.arange(0, 0.91, 0.1):
        p_pl = 1 - p_divida
        w = wacc(ke, kd_bruto, ir, p_pl, p_divida)
        print(f"{p_divida*100:<11.0f}% {p_pl*100:<11.0f}% {w*100:<10.2f}%")
```

analise_wacc()

Dívida (%)	PL (%)	WACC (%)

0%	100%	14.50%
10%	90%	13.69%
20%	80%	12.88%
30%	70%	12.08%
40%	60%	11.27%
50%	50%	10.46%
60%	40%	9.65%
70%	30%	8.84%

80%	20%	8.04%
90%	10%	7.23%

Atenção: O WACC diminui com mais dívida apenas até certo ponto. Com endividamento excessivo, o risco de falência aumenta, elevando tanto K_e quanto K_d .

3.5. Teoria do Custo de Capital

A estrutura de capital ótima é aquela que **minimiza o WACC** e, consequentemente, **maximiza o valor da empresa**.

```
def valor_empresa_modelo(fcf_perpetuo, wacc):
    """Valor da empresa pela perpetuidade do FCF."""
    return fcf_perpetuo / wacc

fcf = 100 # fluxo de caixa livre perpétuo (milhões)
print(f"FCF perpétuo: R${fcf} milhões\n")

for w in [0.08, 0.10, 0.12, 0.14, 0.16]:
    v = valor_empresa_modelo(fcf, w)
    print(f"WACC = {w*100:.0f}% -> Valor = R${v:,.0f} milhões")
```

Capítulo 4 – Valuation (Avaliação de Empresas)

4.1. Abordagens de Valuation

Abordagem	Método	Base
Fluxo de Caixa Descontado	FCD (DCF)	Valor intrínseco (fundamentalista)
Relativa (Múltiplos)	P/L, EV/EBITDA, P/VP	Comparação com pares
Base em Ativos	Valor Patrimonial, Liquidação	Custo de reposição
Base em Opções	Black-Scholes, Binomial	Flexibilidade gerencial

4.2. Fluxo de Caixa Descontado (FCD / DCF)

O valor intrínseco de uma empresa é o valor presente de todos os fluxos de caixa futuros:

$$EV = \sum_{t=1}^n \frac{FCF_t}{(1 + WACC)^t} + \frac{VT}{(1 + WACC)^n}$$

Onde: - EV = Enterprise Value (valor da firma) - FCF_t = Fluxo de Caixa Livre no ano t - $WACC$ = custo médio ponderado de capital - VT = Valor Terminal (perpetuidade)

Valor Terminal (Gordon):

$$VT = \frac{FCF_n \times (1 + g)}{WACC - g}$$

Equity Value:

$$Equity\ Value = EV - Dívida + Caixa$$

```
def valuation_fcd(fcfs, wacc, g, divida, caixa):
    """
    Valuation pelo Fluxo de Caixa Descontado.
```

```

"""
n = len(fcfs)
# VP dos FCFs explícitos
vp_fcf = sum(fcf / (1 + wacc) ** (t + 1) for t, fcf in enumerate(fcfs))

# Valor Terminal
ultimo_fcf = fcfs[-1]
vt = ultimo_fcf * (1 + g) / (wacc - g)
vp_vt = vt / (1 + wacc) ** n

ev = vp_fcf + vp_vt
equity_value = ev - divida + caixa

return {
    'VP_FCFs': round(vp_fcf, 2),
    'VP_Valor_Terminal': round(vp_vt, 2),
    'Enterprise_Value': round(ev, 2),
    '(-) Dívida': round(-divida, 2),
    '(+) Caixa': round(caixa, 2),
    'Equity_Value': round(equity_value, 2)
}

# Exemplo
fcfs = [100, 115, 132, 152, 175]
wacc = 0.12
g = 0.035
divida = 300
caixa = 80

```

```

resultado = valuation_fcd(fcfs, wacc, g, divida, caixa)
print("=== Valuation por FCD ===")
for k, v in resultado.items():
    print(f"{k:25s}: R${v:>10,.2f}")

=== Valuation por FCD ===
VP_FCFs                : R$    538.63
VP_Valor_Terminal      : R$  1,300.27
Enterprise_Value       : R$  1,838.90
(-) Dívida             : R$   -300.00
(+) Caixa              : R$    80.00
Equity_Value           : R$  1,618.90

```

4.3. Valuation por Múltiplos

Mais rápido que o FCD, mas depende de encontrar empresas comparáveis.

```

def multiplos_empresa(lucro, ebitda, receita, valor_patrimonial, num_acoes,
                      pl_setor, ev_ebitda_setor, pvp_setor):
    """
    Estima o preço justo por múltiplos de mercado.
    """

    preco_pl = pl_setor * (lucro / num_acoes)
    preco_pvp = pvp_setor * (valor_patrimonial / num_acoes)

    # Para EV/EBITDA, precisamos estimar o EV
    ev_estimado = ev_ebitda_setor * ebitda

    print(f"'Múltiplo':<15} {'Múltiplo Setor':<18} {'Preço Justo':<15}")

```

```

print("-" * 48)
print(f"{'P/L':<15} {pl_setor:<18.2f}x R\${preco_pl:<10.2f}")
print(f"{'EV/EBITDA':<15} {ev_ebitda_setor:<18.2f}x ---")
print(f"{'P/VP':<15} {pvp_setor:<18.2f}x R\${preco_pvp:<10.2f}")

return {'P/L': preco_pl, 'P/VP': preco_pvp}

# Empresa com LPA = R$3,50; valor patrimonial por ação = R$20
multiplos_empresa(
    lucro=350e6, ebitda=600e6, receita=1.2e9,
    valor_patrimonial=2e9, num_acoes=100e6,
    pl_setor=12, ev_ebitda_setor=7, pvp_setor=1.5
)

```

4.4. Múltiplos Comuns

Múltiplo	Fórmula	Indicação	Melhor Uso
P/L (Preço/Lucro)	P/LPA	Mais popular	Empresas maduras, lucro estável
EV/EBITDA	$EV/EBITDA$	Ignora depreciação	Empresas de capital intensivo
P/VP	P/VPA	Valor patrimonial	Bancos, seguradoras
Div. Yield	DPA/P	Retorno em dividendos	Empresas que distribuem lucro
P/Receita	$P/Receita$	Empresas sem lucro	Startups, crescimento
EV/FCF	EV/FCF	Geração de caixa	Qualquer empresa

Capítulo 5 – Análise de Demonstrações Financeiras

5.1. As Três Demonstrações

1. Balanço Patrimonial (BP) – “Fotografia” Mostra a posição financeira em uma data específica:

$$Ativo = Passivo + PatrimônioLíquido$$

2. Demonstração do Resultado (DRE) – “Filme” Mostra a geração de lucro em um período:

$$Receita - Custos - Despesas = LucroLíquido$$

3. Demonstração do Fluxo de Caixa (DFC) Mostra as origens e usos do caixa, dividido em operacional, investimento e financiamento.

5.2. Análise Vertical e Horizontal

Análise Vertical: cada item é expresso como percentual de uma base (receita total, ativo total).

Análise Horizontal: evolução dos itens ao longo do tempo.

```

def analise_vertical(dre):
    """DRE como percentual da receita líquida."""
    receita = dre['receita_liquida']
    print(f"{'Item':<30} {'Valor (R$ mil)':<18} {'% Receita':<10}")
    print("-" * 58)
    for item, valor in dre.items():
        pct = valor / receita * 100
        print(f"{'item':<30} R\${valor:<13,.0f} {pct:<9.2f}%")

```

```
dre_exemplo = {
    'receita_liquida': 1000000,
    'custo_produtos': -550000,
    'despesas_operacionais': -200000,
    'despesas_financeiras': -50000,
    'imposto_renda': -68000
}
analise_vertical(dre_exemplo)
```

5.3. Indicadores de Liquidez

Medem a capacidade de pagar obrigações de curto prazo.

Indicador	Fórmula	Interpretação
Liquidez Corrente	AC/PC	Ideal > 1,5
Liquidez Seca	$(AC - Estoques)/PC$	Ideal > 1,0
Liquidez Imediata	$Disponível/PC$	Capacidade imediata
Liquidez Geral	$(AC + RLP)/(PC + ELP)$	Longo prazo

```
def indicadores_liquidez(ac, pc, estoques, disponivel, rlp, elp):
    lc = ac / pc
    ls = (ac - estoques) / pc
    li = disponivel / pc
    lg = (ac + rlp) / (pc + elp)
    return {'Corrente': lc, 'Seca': ls, 'Imediata': li, 'Geral': lg}
```

```
# Exemplo
ac, pc, est, disp, rlp, elp = 8000, 5000, 2000, 1000, 3000, 2000
liq = indicadores_liquidez(ac, pc, est, disp, rlp, elp)
for k, v in liq.items():
    status = "OK" if v >= 1.0 else "ATENÇÃO"
    print(f"Liquidez {k}: {v:.2f} [{status}]")
```

5.4. Indicadores de Endividamento

Medem a estrutura de capital e o risco financeiro.

Indicador	Fórmula	Interpretação
Dívida/PL	$Passivo/PL$	Quanto maior, mais alavancado
Dívida/Ativo	$Passivo/Ativo$	Percentual financiado por terceiros
ICJ	$LAJIR/DF$	Cobertura de juros
Composição do Endividamento	$PC/(PC + ELP)$	Perfil da dívida

```
def indicadores_endividamento(passivo_circ, passivo_ncirc, pl, lajir, despesas_fin):
    pt = passivo_circ + passivo_ncirc
    d_pl = pt / pl
    d_ativ = pt / (pt + pl)
    icj = lajir / despesas_fin if despesas_fin != 0 else float('inf')
    comp_end = passivo_circ / pt
    return {'Dívida/PL': d_pl, 'Dívida/Ativo': d_ativ,
            'ICJ': icj, 'Composição CP': comp_end}
```

```
# Exemplo
```

```
ind_end = indicadores_endividamento(5000, 3000, 7000, 1200, 400)
for k, v in ind_end.items():
    print(f"{k}: {v:.2f}")
```

5.5. Indicadores de Rentabilidade

Medem a capacidade de gerar retorno sobre os recursos investidos.

Indicador	Fórmula	Significado
ROE	LL/PL	Retorno do acionista
ROA	$LL/Ativo$	Retorno sobre ativos
ROIC	$NOPAT/CapitalInvestido$	Retorno operacional
Margem Líquida	$LL/Receita$	Lucratividade sobre vendas
Margem Bruta	$(Receita - CPV)/Receita$	Lucro após custo dos produtos
Giro do Ativo	$Receita/Ativo$	Eficiência no uso dos ativos

```
def indicadores_rentabilidade(ll, nopat, receita, ativo, pl, capital_investido):
    return {
        'ROE': ll / pl,
        'ROA': ll / ativo,
        'ROIC': nopat / capital_investido,
        'Margem Líquida': ll / receita,
        'Margem Bruta': 100, # precisaria de CPV
        'Giro do Ativo': receita / ativo
    }
```

```
rent = indicadores_rentabilidade(800, 1100, 15000, 12000, 7000, 9000)
for k, v in rent.items():
    if k in ('Giro do Ativo',):
        print(f"{k}: {v:.2f}x")
    else:
        print(f"{k}: {v*100:.2f}%")
```

5.6. Sistema DuPont

Decompõe o ROE em suas alavancas operacionais e financeiras:

$$ROE = \frac{LL}{Vendas} \times \frac{Vendas}{Ativo} \times \frac{Ativo}{PL}$$

$$ROE = Margem\ Líquida \times Giro\ do\ Ativo \times Alavancagem\ Financeira$$

Utilidade: identifica qual alavanca está puxando (ou prejudicando) o retorno do acionista.

```
def dupont_analysis(ll, vendas, ativo, pl):
    margem = ll / vendas
    giro = vendas / ativo
    alavancagem = ativo / pl
    roe = margem * giro * alavancagem

    print("=== Análise DuPont ===")
    print(f"Margem Líquida: {margem*100:.2f}%")
    print(f"Giro do Ativo: {giro:.2f}x")
    print(f"Alavancagem Financeira: {alavancagem:.2f}x")
    print(f"---")
```

```

print(f"ROE: {margem*100:.2f}% × {giro:.2f}x × {alavancagem:.2f}x =
{roe*100:.2f}%")

# Contribuição de cada alavanca
contrib_margem = margem * 1 * 1
contrib_giro = margem * giro * 1
contrib_alav = margem * giro * alavancagem
print(f"\nContribuição marginal:")
print(f"  Margem isolada: {contrib_margem*100:.2f}%")
print(f"  + Giro: {contrib_giro*100:.2f}%")
print(f"  + Alavancagem: {contrib_alav*100:.2f}%")

dupont_analysis(800, 15000, 12000, 7000)

```

5.7. EBITDA e EBITDA Ajustado

EBITDA (Earnings Before Interest, Taxes, Depreciation and Amortization) — Lucro antes de juros, impostos, depreciação e amortização.

$$EBITDA = LAJIR + Depreciação + Amortização$$

Importante: o EBITDA não é fluxo de caixa, pois ignora: - Investimentos (CapEx) - Necessidade de Capital de Giro (NCG) - Impostos e juros (que são pagos em dinheiro)

```

def calcular_ebitda(lajir, depreciacao, amortizacao):
    return lajir + depreciacao + amortizacao

# DRE simplificada
receita, cpv, despesas_op, deprec, amort = 1000, 600, 150, 80, 30
lajir = receita - cpv - despesas_op
ebitda_ = calcular_ebitda(lajir, deprec, amort)

print(f"Receita: {receita}")
print(f"CPV: -{cpv}")
print(f"Despesas Op: -{despesas_op}")
print(f"= LAJIR (EBIT): {lajir}")
print(f"+ Depreciação: +{deprec}")
print(f"+ Amortização: +{amort}")
print(f"= EBITDA: {ebitda_}")
print(f"Margem EBITDA: {ebitda_/receita*100:.1f}%")

```

Capítulo 6 — Estrutura de Capital

6.1. A Pergunta Fundamental

Existe uma estrutura de capital ótima? Isto é, uma combinação dívida/capital próprio que maximiza o valor da empresa?

Esta é uma das questões mais debatidas em finanças corporativas.

6.2. Modigliani-Miller (1958) — Sem Impostos

Proposição I — Irrelevância da Estrutura de Capital:

Em mercados perfeitos (sem impostos, sem custos de falência, sem assimetria de informação), o valor da empresa **independe** da estrutura de capital.

$$V_L = V_U$$

Proposição II – Custo do Capital Próprio:

O custo do capital próprio aumenta linearmente com o endividamento:

$$K_e = K_{e_U} + (K_{e_U} - K_d) \times \frac{D}{E}$$

Implicação: o aumento do K_e compensa exatamente o benefício da dívida mais barata, mantendo o WACC constante.

```
def mm_sem_impostos(ke_u, kd, d, e):
    """Proposição II de M&M sem impostos."""
    ke = ke_u + (ke_u - kd) * (d / e)
    v = d + e
    wacc_ = (e/v) * ke + (d/v) * kd
    return {'Ke': ke, 'WACC': wacc_}

print("M&M sem impostos:")
for d_e in [0, 0.25, 0.5, 1.0, 2.0]:
    d = d_e * 100
    e = 100
    res = mm_sem_impostos(0.15, 0.10, d, e)
    print(f" D/E = {d_e:.2f}: Ke = {res['Ke']*100:.2f}%, WACC = {res['WACC']*100:.2f}%")
```

6.3. Modigliani-Miller (1963) – Com Impostos

Com a dedutibilidade dos juros da dívida, a empresa alavancada vale **mais**:

$$V_L = V_U + D \times IR$$

O termo $D \times IR$ é o **benefício fiscal** (escudo fiscal) da dívida.

```
def mm_com_impostos(vu, d, ir):
    """M&M com impostos corporativos."""
    beneficio_fiscal = d * ir
    vl = vu + beneficio_fiscal
    return {'Vu': vu, 'Benefício Fiscal': beneficio_fiscal, 'VL': vl}

print("M&M com impostos (IR = 34%):")
for d in [0, 100, 200, 300, 400]:
    res = mm_com_impostos(1000, d, 0.34)
    print(f" Dívida = {d:3d}: VL = R${res['VL']:,.0f} (Benefício = R${res['Benefício Fiscal']:,.0f})")
```

6.4. Trade-off Theory

Na prática, o endividamento excessivo traz **custos de falência** (diretos e indiretos):

$$V_L = V_U + VP(\text{Benefício Fiscal}) - VP(\text{Custo de Falência})$$

Valor da Empresa

```

^
|   * Valor com benefício fiscal (M&M c/ impostos)
|   /|
|   / |
|   /  * Valor real (Trade-off)
|   /   |
|   /    ^
*-----> Endividamento
```


Custos de falência: - **Diretos:** honorários advocatícios, custas judiciais, perícias - **Indiretos:** perda de clientes, fornecedores, funcionários; vendas a preços baixos

A **estrutura de capital ótima** está no ponto em que o benefício marginal da dívida se iguala ao custo marginal esperado de falência.

6.5. Pecking Order Theory (Myers & Majluf, 1984)

Devido à **assimetria de informação** (gestores sabem mais que investidores), as empresas têm uma hierarquia de preferência para financiamento:

1. **Lucros retidos** (recursos internos) — sem custo de seleção adversa
2. **Dívida** — sinal menos negativo que a emissão de ações
3. **Ações** — último recurso (sinal de que a ação pode estar sobrevalorizada)

Previsões da Pecking Order: - Empresas lucrativas se endividam **menos** (têm mais recursos internos) - Empresas com muitas oportunidades de crescimento emitem menos ações - A estrutura de capital é resultado de decisões passadas, não de um alvo

Capítulo 7 — Política de Dividendos

7.1. Tipos de Política

Tipo	Descrição	Exemplo
Constante	Mesmo valor todo período	R\$ 0,50 por ação todo trimestre
Crescente	Aumento regular	Aumento de 5% ao ano no dividendo
Pay-out fixo	Percentual constante do lucro	40% do lucro líquido
Residual	Distribui o que sobra após investir	Só paga se não houver projetos viáveis

7.2. Métricas de Dividendos

$$DPA = \frac{\text{Dividendos Totais}}{\text{Número de Ações}}$$

$$\text{Pay-out} = \frac{\text{Dividendos}}{\text{Lucro Líquido}}$$

$$\text{Dividend Yield} = \frac{DPA}{\text{Preço da Ação}}$$

```
def metricas_dividendos(lucro_liquido, dividendos_pagos, num_acoes, preco_acao):  
    lpa = lucro_liquido / num_acoes  
    dpa = dividendos_pagos / num_acoes  
    pay_out = dividendos_pagos / lucro_liquido  
    dy = dpa / preco_acao  
  
    print(f"{'Métrica':<20} {'Valor'}")  
    print("-" * 30)  
    print(f"{'LPA':<20} R${lpa:.2f}")  
    print(f"{'DPA':<20} R${dpa:.2f}")  
    print(f"{'Pay-out':<20} {pay_out*100:.1f}%")
```

```

print(f"{'Dividend Yield':<20} {dy*100:.2f}%")
print(f"{'Retenção (1-pay-out)':<20} {(1-pay_out)*100:.1f}%")

metricas_dividendos(500e6, 200e6, 100e6, 35)

```

7.3. Teoria da Irrelevância (M&M, 1961)

Em mercados perfeitos, a política de dividendos **não afeta o valor da empresa**. O valor deriva da capacidade de gerar lucro e da política de investimentos, não da forma como o lucro é distribuído.

Argumento: se a empresa retém lucros em vez de distribuir, o preço da ação aumenta compensando o dividendo não recebido. O acionista pode criar seu próprio dividendo vendendo ações.

7.4. Teoria do Pássaro na Mão (Gordon, 1963)

Contraria M&M: investidores preferem dividendos hoje a ganhos de capital futuros (mais arriscados). Portanto, um aumento no pay-out reduz o K_e e aumenta o valor da empresa.

7.5. Efeito Clientela

Diferentes grupos de investidores preferem diferentes políticas de dividendos: - **Aposentados:** preferem alta distribuição (renda) - **Fundos de pensão:** podem preferir retenção (crescimento) - **Empresas:** podem preferir recompra de ações (tributação menor)

Capítulo 8 — Capital de Giro

8.1. Conceitos Básicos

Capital de Giro = recursos necessários para financiar as operações do dia a dia.

$$CCL = \text{Ativo Circulante} - \text{Passivo Circulante}$$

$$NCG = AC \text{ Operacional} - PC \text{ Operacional}$$

$$ST = CCL - NCG$$

Onde: - $AC \text{ Operacional}$ = contas a receber + estoques + adiantamentos - $PC \text{ Operacional}$ = fornecedores + salários + impostos a pagar - ST (Saldo de Tesouraria) > 0 indica folga financeira

```

def diagnostico_capital_giro(ac, pc, ac_op, pc_op):
    ccl = ac - pc
    ncg = ac_op - pc_op
    st = ccl - ncg

    print(f"{'Indicador':<25} {'Valor (R$)'}")
    print("-" * 35)
    print(f"{'Capital Circulante Líquido':<25} R${ccl:,.2f}")
    print(f"{'Necessidade de Capital de Giro':<25} R${ncg:,.2f}")
    print(f"{'Saldo de Tesouraria':<25} R${st:,.2f}")

    if st >= 0:
        print("\nSituação: CONFORTO FINANCEIRO (ST >= 0)")
    elif st >= -ncg * 0.3:
        print("\nSituação: ATENÇÃO (ST negativo moderado)")
    else:
        print("\nSituação: RISCO (ST muito negativo)")

diagnostico_capital_giro(8000, 5000, 6000, 3000)

```

8.2. Ciclos Operacionais

```
def ciclos_economico_financeiro(pme, pmr, pmp):  
    """  
    pme = prazo médio de estocagem (dias)  
    pmr = prazo médio de recebimento (dias)  
    pmp = prazo médio de pagamento (dias)  
    """  
  
    co = pme + pmr # ciclo operacional  
    cf = co - pmp # ciclo financeiro (caixa)  
  
    print(f"Prazo médio de estocagem: {pme} dias")  
    print(f"Prazo médio de recebimento: {pmr} dias")  
    print(f"Prazo médio de pagamento: {pmp} dias")  
    print(f"---")  
    print(f"Ciclo Operacional (CO): {co} dias")  
    print(f"Ciclo Financeiro (CF): {cf} dias")  
  
    if cf > 0:  
        print(f"\nA empresa precisa financiar {cf} dias de operação.")  
    else:  
        print(f"\nA empresa opera com capital de giro negativo (vantagem competitiva).")  
  
# Indústria  
print("=== Indústria ===")  
ciclos_economico_financeiro(60, 45, 30)  
  
# Varejo  
print("\n=== Varejo (supermercado) ===")  
ciclos_economico_financeiro(30, 7, 45)
```

8.3. Capital de Giro Negativo

Algumas empresas operam com **capital de giro negativo** ($CCL < 0$), o que significa que financiam seus ativos de curto prazo com passivos de curto prazo. Exemplos:

- **Supermercados:** vendem à vista, compram a prazo ($PMP > PMR$)
- **Aviação:** vendem passagens antes de pagar combustível e salários
- **Software:** recebem assinaturas antes de incorrer em custos

Capítulo 9 — Fusões e Aquisições (M&A)

9.1. Motivações

Motivo	Descrição
Sinergia	$2 + 2 = 5$ (economias de escala, receitas cruzadas)
Diversificação	Reduzir risco (embora questionável para o acionista)
Poder de mercado	Aumentar participação, eliminar concorrência
Eficiência	Substituir gestão ineficiente
Acesso a tecnologia	Comprar inovação em vez de desenvolver
Benefício fiscal	Usar prejuízos fiscais da empresa alvo

9.2. Sinergias

$$V(AB) > V(A) + V(B)$$

$$\text{Sinergia} = V(AB) - [V(A) + V(B)]$$

```
def analise_sinergia(v_a, v_b, v_ab, premio_pago):
    """Analisa criação de valor em uma fusão."""
    valor_sem_sinergia = v_a + v_b
    sinergia = v_ab - valor_sem_sinergia
    valor_comprador = sinergia - premio_pago

    print(f"Valor A (isolado): R\${v_a:,.0f} M")
    print(f"Valor B (isolado): R\${v_b:,.0f} M")
    print(f"Valor AB (combinado): R\${v_ab:,.0f} M")
    print(f"---")
    print(f"Sinergia total: R\${sinergia:,.0f} M")
    print(f"Prêmio pago aos acionistas de B: R\${premio_pago:,.0f} M")
    print(f"Valor líquido para acionistas de A: R\${valor_comprador:,.0f} M")

    if valor_comprador > 0:
        print("Fusão CRIADORA de valor para A")
    else:
        print("Fusão DESTRUIDORA de valor para A")

analise_sinergia(500, 300, 950, 60)
```

9.3. Tipos de Integração

Horizontal

A (concorrente) + B (concorrente)
Ex: Itaú + Unibanco

Vertical

A (fornecedor) + B (cliente)
Ex: Petrobras + distribuidora

Conglomerado

A + B (setores diferentes)
Ex: GE (vários setores)

9.4. O Processo de M&A

1. **Estratégia:** definir alvos, critérios
2. **Due Diligence:** investigar a empresa alvo (financeira, fiscal, legal, operacional)
3. **Valuation:** quanto vale a empresa alvo?
4. **Negociação:** preço, forma de pagamento (dinheiro, ações), earn-out
5. **Estruturação:** compra de ativos vs compra de ações
6. **Integração:** pós-fusão (culture clash, sistemas, pessoas)

Capítulo 10 — Finanças Comportamentais

10.1. Racionalidade Limitada

A teoria financeira clássica assume que investidores são **racionais** e mercados são **eficientes**. As finanças comportamentais relaxam essas premissas, incorporando insights da psicologia.

10.2. Principais Vieses

Viés	Descrição	Efeito no Mercado
Excesso de Confiança	Superestimamos nossa capacidade	Volume de negociação excessivo
Aversão a Perda	Perder dói 2x mais que ganhar prazer	Disposição effect (vender winners, segurar losers)
Ancoragem	Prender-se a um preço de referência	Sub-reação a novas informações
Viés de Confirmação	Buscar o que confirma nossas crenças	Ignorar sinais contrários
Efeito Manada	Seguir a multidão	Bolhas e crashes
Framing	Decisão depende de como é apresentada	Preferências inconsistentes
Falácia do Custo Afundado	Deixar custos passados influenciarem	Segurar posições perdedoras

10.3. Anomalias de Mercado

Anomalias são padrões de retorno que contradizem a Hipótese de Mercado Eficiente (EMH):

```
anomalias = {  
    'Efeito Janeiro': 'Retornos anormais em janeiro, especialmente small caps',  
    'Efeito Segunda-feira': 'Retornos negativos nas segundas-feiras',  
    'Momentum': 'Ações que subiram nos últimos 6-12 meses continuam subindo',  
    'Reversão': 'Ações que perderam muito nos últimos 3-5 anos se recuperam',  
    'Valor': 'Baixo P/L e P/VP geram retornos superiores (Fama-French)',  
    'Tamanho': 'Small caps têm retorno superior ajustado ao risco',  
    'Baixo Risco': 'Ações de baixa volatilidade têm retornos superiores  
(paradoxalmente)',  
    'IPO': 'IPOs têm performance inferior no longo prazo'  
}
```

```
print("=== Anomalias de Mercado ===")  
for nome, desc in anomalias.items():  
    print(f"{nome:<20} -> {desc}")
```

Capítulo 11 — Mercados Financeiros

11.1. Estrutura

Sistema Financeiro Nacional

- └─ Órgãos Reguladores (CMN, BACEN, CVM, SUSEP, PREVIC)
- └─ Operadores
 - └─ Bancos Múltiplos, Comerciais, de Investimento
 - └─ Corretoras e Distribuidoras
 - └─ Seguradoras e Entidades de Previdência
 - └─ Fundos de Investimento
- └─ Mercados
 - └─ Mercado Monetário (curto prazo, títulos públicos)
 - └─ Mercado de Crédito (empréstimos bancários)

- └ Mercado de Capitais (ações, debêntures, FIIs)
- └ Mercado de Câmbio (moedas estrangeiras)

11.2. Renda Fixa

Títulos públicos federais (Tesouro Direto): - **Tesouro Selic (LFT):** pós-fixado (Selic) - **Tesouro Prefixado (LTN):** taxa definida na emissão - **Tesouro IPCA+ (NTN-B):** IPCA + taxa real

Títulos privados: - **CDB:** Certificado de Depósito Bancário - **RDB:** Recibo de Depósito Bancário - **LCI/LCA:** Letra de Crédito Imobiliário/Agronegócio (isento IR PF) - **Debêntures:** dívida de empresas não financeiras - **CRI/CRA:** Certificado de Recebível Imobiliário/Agronegócio

```
def calcular_tesouro_ipca(valor, vencimento_anos, taxa_real, ipca_projetado):
    """Calcula o valor de resgate de um título IPCA+."""
    taxa_total = (1 + taxa_real) * (1 + ipca_projetado) - 1
    vf = valor * (1 + taxa_total) ** vencimento_anos
    vf_real = valor * (1 + taxa_real) ** vencimento_anos
    return {'Valor Nominal': vf, 'Valor Real': vf_real, 'Taxa Total': taxa_total}

# R$1.000 em Tesouro IPCA+ com taxa real de 5,5% a.a., IPCA projetado 4% a.a., 5 anos
res = calcular_tesouro_ipca(1000, 5, 0.055, 0.04)
for k, v in res.items():
    if 'Taxa' in k:
        print(f"{k}: {v*100:.2f}%")
    else:
        print(f"{k}: R${v:.2f}")
```

11.3. Renda Variável

Ações: - **ON (Ordinária):** com direito a voto - **PN (Preferencial):** preferência no recebimento de dividendos, sem voto - **Units:** conjunto de ON + PN negociado como um ativo

Derivativos: - **Opções:** direito de comprar (call) ou vender (put) a um preço fixo - **Futuros:** obrigação de comprar/vender no futuro - **Swaps:** troca de fluxos financeiros (ex: CDI x IPCA)

11.4. Análise de um Ativo

```
def analisar_acao(ticker, periodo="5y"):
    """
    Análise fundamentalista básica de uma ação.
    """
    dados = yf.download(ticker, period=periodo)
    precos = dados['Adj Close']
    retornos = precos.pct_change().dropna()

    preco_atual = precos.iloc[-1]
    preco_min = precos.min()
    preco_max = precos.max()
    retorno_anual = (preco_atual / precos.iloc[0]) ** (252 / len(precos)) - 1
    volatilidade = retornos.std() * np.sqrt(252)

    # Drawdown máximo
    pico = precos.expanding().max()
    drawdown = (precos - pico) / pico
    max_drawdown = drawdown.min()

    print(f"=== Análise de {ticker} ===")
    print(f"Período: {periodo}")
    print(f"Preço atual: R${preco_atual:.2f}")
```

```

print(f"Mínimo do período: R${preco_min:.2f}")
print(f"Máximo do período: R${preco_max:.2f}")
print(f"Retorno anualizado: {retorno_anual*100:.2f}%")
print(f"Volatilidade anual: {volatilidade*100:.2f}%")
print(f"Drawdown máximo: {max_drawdown*100:.2f}%")
print(f"Retorno/Vol (Sharpe): {retorno_anual/volatilidade:.2f}")

# analisar_acao("PETR4.SA", "3y")

```

Capítulo 12 — Finanças Internacionais

12.1. Taxa de Câmbio

Taxa de câmbio = preço de uma moeda em termos de outra.

$$R\$/US\$ = \frac{R\$}{US\$}$$

Regimes cambiais: - **Fixo:** governo define a taxa - **Flutuante:** mercado define (Brasil adota este) -

Banda cambial: flutuação dentro de limites

```

def paridade_descoberta_juros(taxa_juros_br, taxa_juros_usa, taxa_cambio_spot,
    periodo):
    """
    Taxa de câmbio futura esperada pela paridade descoberta de juros.
    """
    taxa_cambio_futura = taxa_cambio_spot * ((1 + taxa_juros_br) / (1 +
taxa_juros_usa))
    return taxa_cambio_futura

spot = 5.20 # R$/US$
j_br = 0.1325 # Selic
j_usa = 0.05 // Fed Funds
futuro = paridade_descoberta_juros(j_br, j_usa, spot, 1)
print(f"Câmbio spot: R${spot:.2f}")
print(f"Juros Brasil: {j_br*100:.1f}%")
print(f"Juros EUA: {j_usa*100:.1f}%")
print(f"Câmbio futuro esperado (1 ano): R${futuro:.2f}")
print(f"Desvalorização esperada: {(futuro/spot - 1)*100:.2f}%")

```

12.2. Risco-País (EMBI+)

Prêmio de risco que o mercado exige para investir em títulos de um país. Medido pelo spread dos títulos soberanos sobre os Treasuries americanos.

12.3. Teoria da Paridade do Poder de Compra (PPP)

$$Taxa\ de\ Câmbio = \frac{Nível\ de\ Preços\ (País\ A)}{Nível\ de\ Preços\ (País\ B)}$$

PPP Relativa: a variação cambial reflete o diferencial de inflação:

$$\frac{E_t}{E_{t-1}} = \frac{1 + \pi_{doméstica}}{1 + \pi_{estrangeira}}$$

Capítulo 13 – Tabela Resumo de Fórmulas

Conceito	Fórmula	Uso
Retorno Total	$R = (P_t - P_{t-1} + D_t)/P_{t-1}$	Performance
CAPM	$E(R_i) = R_f + \beta_i \times (R_m - R_f)$	Custo de capital próprio
Beta	$\beta_i = Cov(R_i, R_m)/Var(R_m)$	Risco sistemático
Retorno Carteira	$E(R_p) = \sum w_i \times E(R_i)$	Portfólio
Risco Carteira (2 ativos)	$\sigma_p^2 = w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + 2w_1w_2\sigma_1\sigma_2\rho_{12}$	Diversificação
WACC	$WACC = (E/V) \times Ke + (D/V) \times Kd \times (1 - IR)$	TMA para projetos
Ke (Gordon)	$Ke = D_1/P_0 + g$	Ações que pagam dividendos
Kd líquido	$Kd_{liq} = Kd_{bruto} \times (1 - IR)$	Custo da dívida
EVA	$EVA = NOPAT - (Capital \times WACC)$	Criação de valor
ROE	$ROE = LL/PL$	Rentabilidade do acionista
DuPont	$ROE = (LL/V) \times (V/A) \times (A/PL)$	Decomposição ROE
Valuation FCD	$EV = \sum FCF_t/(1 + WACC)^t + VT/(1 + WACC)^n$	Valor intrínseco
Valor Terminal	$VT = FCF_n \times (1 + g)/(WACC - g)$	Perpetuidade
P/L	$PL = Preço/LPA$	Múltiplo
MM I (s/ imposto)	$V_L = V_U$	Irrelevância
MM II (c/ imposto)	$V_L = V_U + D \times IR$	Benefício fiscal
Dividend Yield	$DY = DPA/Preço$	Retorno em dividendos
Pay-out	$PO = Dividendos/LL$	Distribuição de lucro
CCL	$AC - PC$	Capital de giro
Fisher	$(1 + i_n) = (1 + i_r) \times (1 + \pi)$	Inflação
PPP	$\Delta E = (1 + \pi_{dom})/(1 + \pi_{est})$	Câmbio

Capítulo 14 – Glossário

Termo	Definição
CAPM	Modelo de precificação de ativos que relaciona retorno esperado ao risco sistemático
Beta	Medida de sensibilidade de um ativo ao mercado
WACC	Custo médio ponderado de todas as fontes de capital
VPL (NPV)	Soma dos fluxos de caixa descontados pela TMA
TIR (IRR)	Taxa de desconto que zera o VPL
EVA	Lucro econômico: NOPAT menos custo do capital
ROE	Retorno sobre o patrimônio líquido
ROIC	Retorno sobre o capital investido
EBITDA	Lucro antes de juros, impostos, depreciação e amortização
FCF	Fluxo de caixa livre disponível para acionistas e credores
CCL	Capital circulante líquido (AC - PC)
NCG	Necessidade de capital de giro
Múltiplo	Indicador relativo de valuation (P/L, EV/EBITDA)
Alavancagem	Uso de capital de terceiros para ampliar retornos
Sinergia	Valor extra criado pela combinação de empresas
EMH	Hipótese de Mercado Eficiente – preços refletem toda informação
Due Diligence	Processo de investigação pré-aquisição
Pay-out	Percentual do lucro distribuído como dividendos
Gordon	Modelo de valuation por dividendos com crescimento
Fronteira Eficiente	Conjunto de carteiras ótimas risco-retorno
Hedge	Proteção contra movimentos adversos de preço
Derivativo	Ativo cujo valor deriva de outro ativo (opção, futuro, swap)